Advanced Machine Learning, ETH Zurich

Attention

How a word reads the words around it

Carlos Cotrini

Autumn semester 2026

Entry 1

Words in context



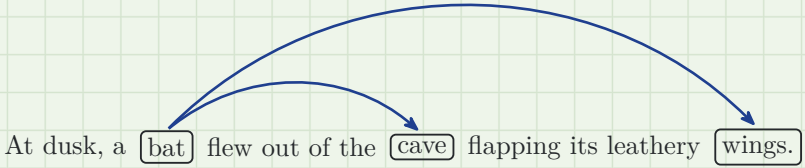
The question. In the language model, every row of E holds one word, yet the next word depends on the whole prefix. So row i must read the rows above it. How?

Now. Attention: every row reads the other rows and adds to itself what it finds. In Wuggish, the toy language of this lecture, every word finds what it needs in the rows above it.

The same word can mean two different things. The words around it decide which one.

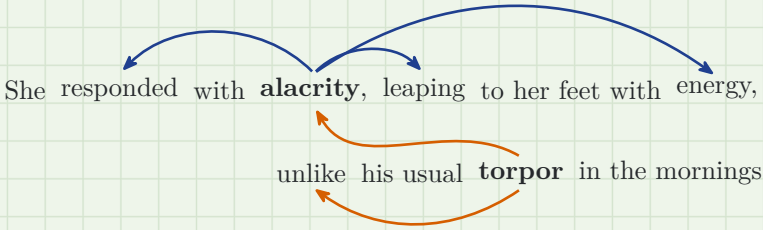


Here **bat** reads baseball and hit: a club.



Here **bat** reads cave and wings: an animal.

Few people know the words alacrity and torpor. Read the sentence: its other words tell you what they mean.



alacrity reads responded, leaping and energy: something quick and lively.

torpor reads unlike and alacrity: the opposite of alacrity.



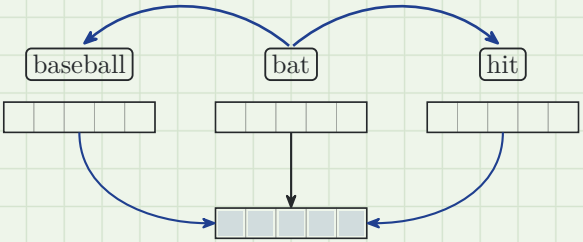
alacrity: brisk and cheerful readiness.



torpor: a state of physical or mental inactivity.

A network that reads a sentence does it in two steps, and every word does both.

1. Every word gets a representation: a row of numbers.



bat, now the club

2. Every word reads the other words and adds to its row what it finds.

This is *attention*. The rest of the lecture computes it by hand, on a language small enough to see every number.

You go shopping for a party. What you look for is the *query*; the signs over the shops are the *keys*; what you carry home are the *values*.

Query



Keys

GreenChoice Supermarket



StoryStack Books



Threadline Clothing Co



Candy Cave



Spirit Cellar Liquors



Values













The party matches three keys. Their values go home.

Entry 2

Wuggish



So far. A word takes its meaning from the words around it. Attention lets every word read the other words and add what it finds to its own row.

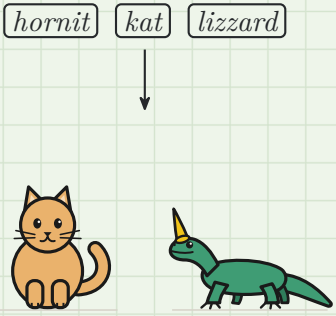
Now. A toy language of eleven words, Wuggish, and a way to write each word as numbers: a row of nine features, each a 1 or a 0.

Wuggish is a made-up sister language of English. Its sentences describe a cat, a pony and a lizard, each plain or with a look: stripes, spots, wings or a horn.

		streept	*spottet*	*winget*	*hornit*
kat			 ***lepperd***		
poony		 ***zeebra***			 ***unikorn***
lizzard				 ***draggon***	

Eleven words: three things, four looks, and four things with a look. *lepperd* is the spotted cat that *spottet kat* draws.

Learn the eleven words from pictures, then read 18 sentences whose words come in a fresh order every time. At the end, the game shows what you have been doing all along.



Play it now, on a laptop or a phone: scan the code, or open the address below.

eth-aml-hs26.github.io/neural-networks/viz/wuggish-game

To compute with words, we ask every word the same nine questions. Each question is a *feature*, and its answer is 1 for yes and 0 for no.

What a thing has

FUR has fur?




HOOVES has hooves?



SCALES has scales?



The nouns say yes here: a cat has fur, a pony fur and hooves, a lizard scales.

Where a look goes

COAT goes on a coat?

BODY goes on a body?

The adjectives say yes here: *streept* and *spottet* go on a coat of fur, *winget* and *hornit* on a body.

What a look brings

STRIPES brings stripes?



SPOTS brings spots?



WINGS brings wings?



HORN brings a horn?



An adjective brings its look, and so does a word that names a thing with a look.

Ask every Wuggish word the nine questions and write its answers in a row, its *embedding*. The eleven rows form the *embedding table*.

	has			goes		brings			
	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
kat	**1**	0	0	0	0	0	0	0	0
poony	**1**	**1**	0	0	0	0	0	0	0
lizzard	0	0	**1**	0	0	0	0	0	0
zeebra	**1**	**1**	0	0	0	**1**	0	0	0
lepperd	**1**	0	0	0	0	0	**1**	0	0
draggon	0	0	**1**	0	0	0	0	**1**	0
unikorn	**1**	**1**	0	0	0	0	0	0	**1**
streept	0	0	0	**1**	0	**1**	0	0	0
spottet	0	0	0	**1**	0	0	**1**	0	0
winget	0	0	0	0	**1**	0	0	**1**	0
hornit	0	0	0	0	**1**	0	0	0	**1**

11 words by $d = 9$ features

kat, *poony* and *lizzard* say what they have: fur; fur and hooves; scales.

zeebra, *lepperd*, *draggon* and *unikorn* each add one look: a zebra is a pony with stripes.

An adjective says where its look goes and what it brings.

We wrote these rows by hand, to read them. A trained network learns its own rows, and its features have no names.

A word of a sentence, a *token*, is a row of the table. A feature is a column: a question that several words answer with yes.

	has			goes		brings			
	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
kat	**1**	0	0	0	0	0	0	0	0
poony	**1**	**1**	0	0	0	0	0	0	0
lizzard	0	0	**1**	0	0	0	0	0	0
zeebra	**1**	**1**	0	0	0	**1**	0	0	0
lepperd	**1**	0	0	0	0	0	**1**	0	0
draggon	0	0	**1**	0	0	0	0	**1**	0
unikorn	**1**	**1**	0	0	0	0	0	0	**1**
streept	0	0	0	**1**	0	**1**	0	0	0
spottet	0	0	0	**1**	0	0	**1**	0	0
winget	0	0	0	0	**1**	0	0	**1**	0
hornit	0	0	0	0	**1**	0	0	0	**1**
words	**5**	**3**	**2**	**2**	**2**	**2**	**2**	**2**	**2**

A token: the row of *lepperd*, nine numbers that say a cat with spots.

A feature: the column FUR. Five words say yes, *kat*, *poony*, *zeebra*, *lepperd* and *unikorn*: fur is what they share.

Every column holds the 1s of two words or more: the count under it.

And there are more words than features: 11 rows, 9 columns. So a feature can never be a second name for a word.

Two rows of the table differ in one column. The row of *lepperd* holds the row of *kat* and one more 1, at SPOTS.

	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
row of *kat*	**1**	0	0	0	0	0	0	0	0
+ SPOTS	0	0	0	0	0	0	**1**	0	0
row of *lepperd*	**1**	0	0	0	0	0	**1**	0	0

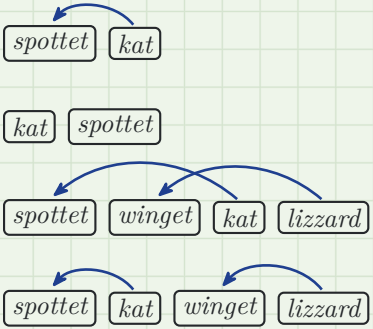




The other three alike: *zeebra* is *poony* plus STRIPES, *draggon* is *lizzard* plus WINGS, and *unikorn* is *poony* plus HORN.

Keep the row of *lepperd* in mind: attention will arrive at it from two words.

Wuggish has one rule of order: an adjective stands somewhere before the noun it describes. Other words may stand between them, and beyond that the order carries nothing.





So a noun finds its adjectives among the words before it: in E, in the rows above its own.

Write the running sentence one word per row and replace every word by its row of the table. The result is E, with $n = 4$ rows and $d = 9$ columns.

	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
spottet	0	0	0	**1**	0	0	**1**	0	0
kat	**1**	0	0	0	0	0	0	0	0
winget	0	0	0	0	**1**	0	0	**1**	0
lizzard	0	0	**1**	0	0	0	0	0	0

E: $n = 4$ rows, $d = 9$ columns

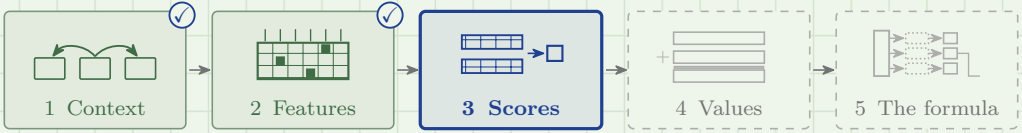
The row of *kat* holds FUR alone: a cat. Nothing in it says spotted, and *spottet* stands in the row above. Attention will make the row of *kat* read the row of *spottet*.

	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
kat	**1**	0	0	0	0	0	0	0	0
poony	**1**	**1**	0	0	0	0	0	0	0
lizzard	0	0	**1**	0	0	0	0	0	0
zeebra	**1**	**1**	0	0	0	**1**	0	0	0
lepperd	**1**	0	0	0	0	0	**1**	0	0
draggon	0	0	**1**	0	0	0	0	**1**	0
unikorn	**1**	**1**	0	0	0	0	0	0	**1**
streept	0	0	0	**1**	0	**1**	0	0	0
spottet	0	0	0	**1**	0	0	**1**	0	0
winget	0	0	0	0	**1**	0	0	**1**	0
hornit	0	0	0	0	**1**	0	0	0	**1**

the embedding table

Entry 3

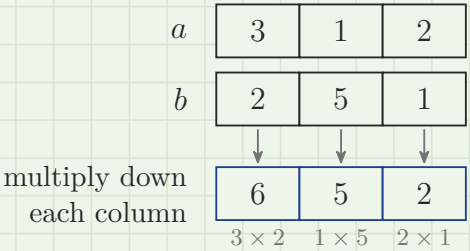
Queries, keys and scores



So far. Every word is a row of nine features, and the running sentence is a block E. In E, the row of *kat* says nothing about spots.

Now. Every word asks with a *query* and answers with a *key*. One number, the inner product of a query and a key, says whether two words go together.

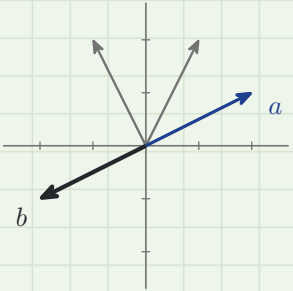
Two rows of the same length give one number, their *inner product*.



then add $6 + 5 + 2 = \boxed{13}$ the inner product of a and b

$$\text{inner product of } a \text{ and } b = a_1b_1 + a_2b_2 + a_3b_3$$

Read a row of two numbers as an arrow. Keep a and turn b: the inner product falls as the angle between them opens.



$a = (2, 1)$

$b = (2, 1)$	$2 \times 2 + 1 \times 1 = 5$	same direction
$b = (1, 2)$	$2 \times 1 + 1 \times 2 = 4$	close directions
$b = (-1, 2)$	$2 \times (-1) + 1 \times 2 = 0$	at right angles
$b = (-2, -1)$	$2 \times (-2) + 1 \times (-1) = -5$	opposite directions

Every word gets a second row, its *query*: what it looks for in the other words. A query has $k = 2$ entries of its own, slot 1 and slot 2.

	FUR	HOOVES	SCALES
kat	1	0	0
poony	1	1	0
lizzard	0	0	1
zeebra	1	1	0
lepperd	1	0	0
draggon	0	0	1
unikorn	1	1	0
streept	0	0	0
spottet	0	0	0
winget	0	0	0
hornit	0	0	0

what a word has

1	2
1	0
1	1
0	1
1	1
1	0
0	1
1	1
0	0
0	0
0	0
0	0

Q

11×2

slot 1 = FUR. A thing with fur looks for a look that goes on a coat.

slot 2 = HOOVES + SCALES. A thing with hooves or scales looks for a look that goes on a body.

An adjective has none of these, so it asks for nothing: its query is $(0, 0)$.

The slots are numbered: they live in queries and keys, apart from the $d = 9$ features.

Every word gets a third row, its *key*: what it offers to the words that look. A key has the same two slots as a query.

	COAT	BODY
kat	0	0
poony	0	0
lizzard	0	0
zeebra	0	0
lepperd	0	0
draggon	0	0
unikorn	0	0
streept	**1**	0
spottet	**1**	0
winget	0	**1**
hornit	0	**1**

where a look goes

	1	**2**
kat	0	0
poony	0	0
lizzard	0	0
zeebra	0	0
lepperd	0	0
draggon	0	0
unikorn	0	0
streept	**1**	0
spottet	**1**	0
winget	0	**1**
hornit	0	**1**

K

11×2

slot 1 = COAT. *streept* and *spottet* go on a coat, so they answer in slot 1.

slot 2 = BODY. *winget* and *hornit* go on a body, so they answer in slot 2.

A noun goes on nothing: its key is $(0, 0)$, also for *zeebra*, *lepperd*, *draggon* and *unikorn*.

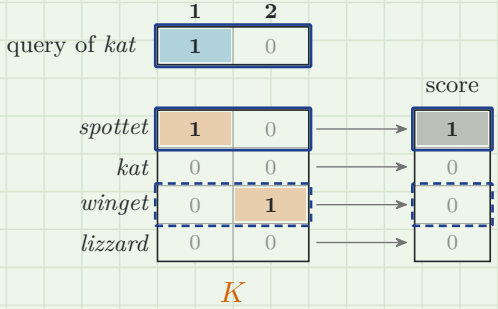
So slot 1 pairs things with fur and looks for a coat, slot 2 things with hooves or scales and looks for a body.

Take the query of *kat* and the key of an adjective. Multiply slot by slot and add: every slot where both rows hold a 1 adds 1.

	1	2	inner product	
query of *kat*	1	0		
key of *streept*	1	0	1	$1 \times 1 + 0 \times 0 = 1$: both fill slot 1
key of *spottet*	1	0	1	the same
key of *winget*	0	1	0	$1 \times 0 + 0 \times 1 = 0$: no slot in common
key of *hornit*	0	1	0	the same

So a cat takes stripes or spots and never wings or a horn. The same sums give a lizard wings or a horn, and a pony all four: the grammar of Wuggish.

Lay the query of *kat* over every key of the running sentence and take the inner products. Each gives a *score*: how well that word fits *kat*.



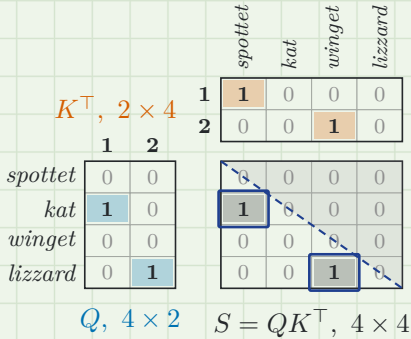
spottet, the word above *kat*, goes on a coat, and a cat has fur: score 1.

winget, the word after *kat*, goes on a body, and a cat takes no wings: score 0.

kat and *lizzard* are nouns, and their keys are all zeros: score 0.

score of *kat* with word j = query of *kat* $\cdot$ key of word j

Stack the queries into Q and lay the keys on their side, $K^\top$. Their product holds every score at once: row i, column j is the query of word i against the key of word j.



Row *kat*, column *spottet*: the query of *kat* against the key of *spottet*, score 1.

Two 1s, one for each noun. An adjective asks for nothing, so its row is 0.

Both 1s lie below the diagonal: a noun reads a word above it, since an adjective stands before its noun.

Attention as computed here reads every row. A language model masks the shaded scores, where a row would read a row below it: in Wuggish they are all 0, so the mask changes nothing here. It comes with the transformer.

Entry 4

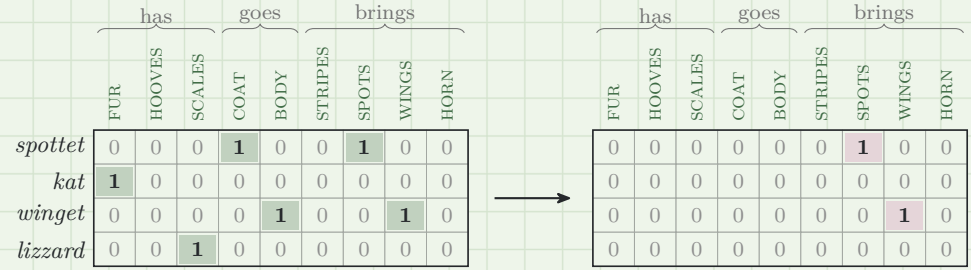
Values



So far. The scores pair *kat* with *spottet* and *lizzard* with *winget*. Each noun has found its adjective, in a row above its own.

Now. Every word gives a *value*, what its look brings, and every word adds to its own row the values of the words it scored 1 with.

Every word gets a fourth row, its *value*: what it adds to a word that reads it. A value keeps the four features of what a look brings and sets the other five to 0.



E, 4×9 V, 4×9

spottet gives SPOTS and *winget* gives WINGS: the brings part of their rows. Where a look goes stays out of the value; the key used it.

A plain noun brings no look: the values of *kat* and *lizzard* are all zeros.

kat adds to its row the value of every word of the sentence, each times its score. Only *spottet* scored 1.

	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
row of *kat* in E	**1**	0	0	0	0	0	0	0	0
$+ 1 \times$ value of *spottet*	0	0	0	0	0	0	**1**	0	0
$+ 0 \times$ every other value	0	0	0	0	0	0	0	0	0
new row of *kat*	**1**	0	0	0	0	0	**1**	0	0

FUR and SPOTS: a cat with spots. Every number is still a 0 or a 1.

new row of *kat* = row of *kat* $+ \sum_j$ score of *kat* with word $j \times$ value of word j

Set the new row of *kat* under the row that the table gives *lepperd*, and compare them column by column.

	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
new row of *kat* after attention	1	0	0	0	0	0	1	0	0
	=	=	=	=	=	=	=	=	=
row of *lepperd* from the table	1	0	0	0	0	0	1	0	0



the same



The same nine numbers, and the same picture: a cat with spots.

A word is a row, and a phrase can arrive at the same row. After one layer of attention, *spottet kat* reads as *lepperd*.

The other noun does the same. *lizzard* scored 1 with *winget*, the word above it, and adds its value, WINGS.

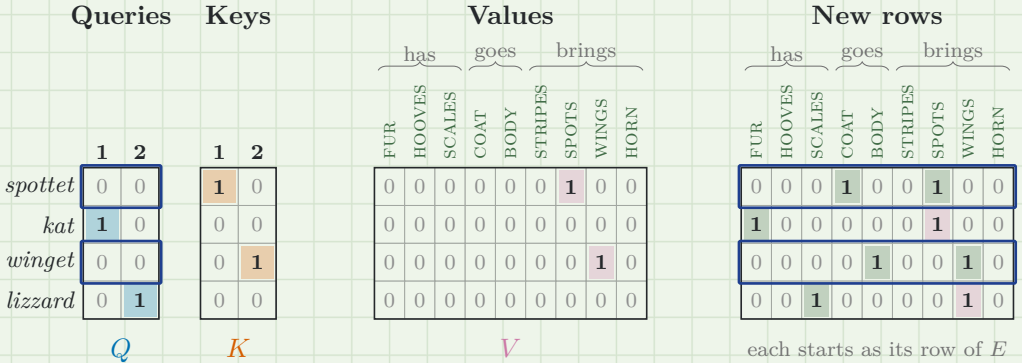
	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
row of *lizzard* in E	0	0	**1**	0	0	0	0	0	0
$+\ 1 \times$ value of *winget*	0	0	0	0	0	0	0	**1**	0
new row of *lizzard*	0	0	**1**	0	0	0	0	**1**	0
=	=	=	=	=	=	=	=	=	=
row of *draggon*	0	0	**1**	0	0	0	0	**1**	0





SCALES and WINGS: after attention, *winget lizzard* reads as *draggon*, the winged lizard.

The running sentence on one sheet: its queries, keys and values side by side. Then one word at a time: its scores, and the values it adds to its own row.

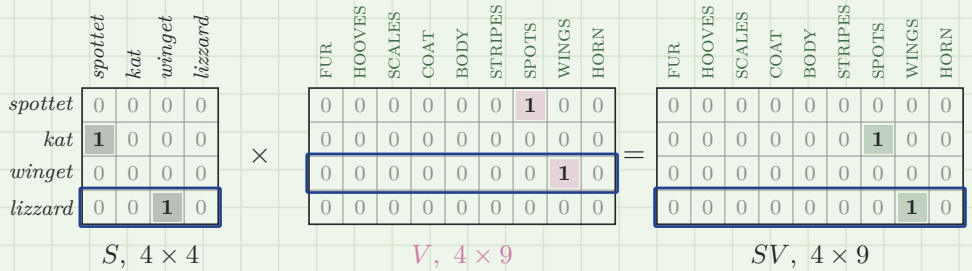


kat adds SPOTS to its own row: FUR and SPOTS, the row of *lepperd*.

lizzard adds WINGS, the value of *winget*: SCALES and WINGS, the row of *draggon*.

An adjective asks for nothing: its query is $(0, 0)$, every score is 0, and its row stays as it was.

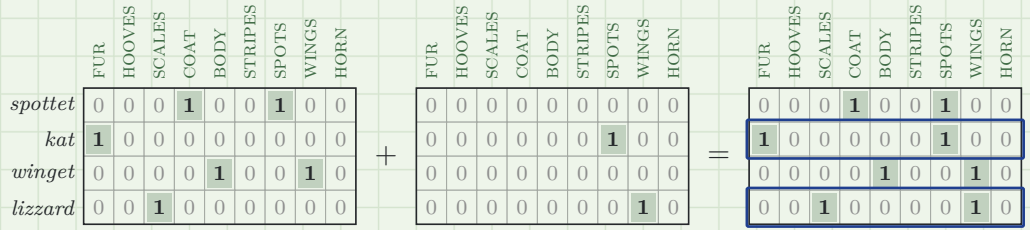
Row i of SV adds up the rows of V, each times its score in row i of S. One product does it for every word at once.



Row *kat* of S holds a 1 at *spottet*, so row *kat* of SV is the value of *spottet*: SPOTS.

Row *lizzard* of S holds a 1 at *winget*, so row *lizzard* of SV is WINGS.

Add SV to E, row by row. A noun keeps its own features and gains the look of its adjective. An adjective found nothing, so it keeps its row as it was.



E SV $E + SV$, 4×9

The two nouns now carry the looks of their adjectives, and every number is still 0 or 1.

Adding E back, so that every word keeps its own row, is the *residual connection*.

Read the new rows of *kat* and *lizzard* against the table. One layer of attention has read the sentence: each adjective has found its noun.

	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN	
spottet	0	0	0	**1**	0	0	**1**	0	0	unchanged
kat	**1**	0	0	0	0	0	**1**	0	0	the row of *lepperd*
winget	0	0	0	0	**1**	0	0	**1**	0	unchanged
lizzard	0	0	**1**	0	0	0	0	**1**	0	the row of *draggon*

$E + SV$

before: the rows of E



after: the rows of $E + SV$



One layer of attention: n rows in, n rows out, each row enriched by the rows it scored 1 with. *spottet kat winget lizzard* now reads as *lepperd* and *draggon*.

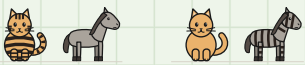
In *streept kat poony* one of the two things is striped: a striped cat and a pony, or a cat and a zebra. Compute the scores and the new rows.

	streept	*kat*	*poony*
streept	0	0	0
kat	**1**	0	0
poony	**1**	0	0

$S,\ 3 \times 3$

Both nouns have fur, so both score 1 with *streept*: a tie.

what a reader means, one of



what attention draws



	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN	
kat	**1**	0	0	0	0	**1**	0	0	0	no word
poony	**1**	**1**	0	0	0	**1**	0	0	0	the row of *zeebra*

new rows of $E + SV$: both striped

Nothing in the words settles it, and attention sees no order at all. The transformer adds the position of every word to its row.

Could the keys be read off the looks themselves, stripes and spots in slot 1, wings and a horn in slot 2? Try *zeebra kat*, a zebra and a cat.

Keys read off the looks

	1	2
key of *zeebra*	1	0

It is the key of *streept*. The query of *kat* scores 1 with it, and *kat* takes the value of *zeebra*, STRIPES:



a zebra and a striped cat

Keys from where a look goes

	1	2
key of *zeebra*	0	0

It is all zeros: a noun goes on nothing. *kat* reads nothing, and both rows stay as they were:



a zebra and a cat

zeebra carries stripes and must keep them: only the adjectives say where a look goes.

Entry 5

Attention in one formula



So far. We wrote every query, key and value by hand, and every new row came out as a 0 or a 1.

Now. A network computes Q, K and V from E with three blocks of weights, and turns every row of scores into weights that add up to 1.

A network does not get the queries for free: it computes them from E, with a block of weights W^Q. Here every weight is a 0 or a 1, so we can read them.

	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
spottet	0	0	0	**1**	0	0	**1**	0	0
kat	**1**	0	0	0	0	0	0	0	0
winget	0	0	0	0	**1**	0	0	**1**	0
lizzard	0	0	**1**	0	0	0	0	0	0

E, 4×9

$\times$

	1	2
FUR	**1**	0
HOOVES	0	**1**
SCALES	0	**1**
COAT	0	0
BODY	0	0
STRIPES	0	0
SPOTS	0	0
WINGS	0	0
HORN	0	0

W^Q, 9×2

$=$

1	2
0	0
1	0
0	0
0	**1**

Q, 4×2

Row FUR of W^Q holds a 1 in slot 1, rows HOOVES and SCALES a 1 in slot 2: the rules of the queries, as a block of weights. The row of *kat* has fur, so it asks in slot 1.

The keys come from E the same way, with a second block of weights, W^K. Its rows say in which slot each feature answers.

	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
spottet	0	0	0	**1**	0	0	**1**	0	0
kat	**1**	0	0	0	0	0	0	0	0
winget	0	0	0	0	**1**	0	0	**1**	0
lizzard	0	0	**1**	0	0	0	0	0	0

E, 4×9

$\times$

	1	**2**
FUR	0	0
HOOVES	0	0
SCALES	0	0
COAT	**1**	0
BODY	0	**1**
STRIPES	0	0
SPOTS	0	0
WINGS	0	0
HORN	0	0

W^K, 9×2

$=$

1	**2**
1	0
0	0
0	**1**
0	0

K, 4×2

Training must find such weights in the text.

Row COAT, slot 1: a look that goes on a coat answers where things with fur ask. Row BODY, slot 2: where things with hooves or scales ask. So a cat takes stripes or spots, a lizard wings or a horn, and a pony all four.

A third block of weights, W^V, makes the values. It keeps the four features of what a look brings and drops the other five.

	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
spottet	0	0	0	**1**	0	0	**1**	0	0
kat	**1**	0	0	0	0	0	0	0	0
winget	0	0	0	0	**1**	0	0	**1**	0
lizzard	0	0	**1**	0	0	0	0	0	0

E, 4×9

$\times$

	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
FUR	0	0	0	0	0	0	0	0	0
HOOVES	0	0	0	0	0	0	0	0	0
SCALES	0	0	0	0	0	0	0	0	0
COAT	0	0	0	0	0	0	0	0	0
BODY	0	0	0	0	0	0	0	0	0
STRIPES	0	0	0	0	0	**1**	0	0	0
SPOTS	0	0	0	0	0	0	**1**	0	0
WINGS	0	0	0	0	0	0	0	**1**	0
HORN	0	0	0	0	0	0	0	0	**1**

W^V, 9×9

$=$

FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
0	0	0	0	0	0	**1**	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	**1**	0
0	0	0	0	0	0	0	0	0

V, 4×9

A 1 on the diagonal at STRIPES, SPOTS, WINGS and HORN, and 0 everywhere else: the value of *spottet* is SPOTS, the value of a plain noun all zeros. These are the values we used.

A real attention layer divides every row of scores by its total. In the running sentence each row that asks reads one word, so nothing changes. Here is a row that reads two.

winget hornit poony

	winget	*hornit*	*poony*		*winget*	*hornit*	*poony*	
scores of *poony*	**1**	**1**	0	$\div 2$	$\frac{1}{2}$	$\frac{1}{2}$	0	weights: they add up to 1

	FUR	HOOVES	SCALES	COAT	BODY	STRIPES	SPOTS	WINGS	HORN
new row, with the scores	**1**	**1**	0	0	0	0	0	**1**	**1**
new row, with the weights	**1**	**1**	0	0	0	0	0	$\frac{1}{2}$	$\frac{1}{2}$



With weights the pony takes half of each look: the same features, at half strength. We kept the raw scores so that every number stayed a 0 or a 1. A row of zeros stays 0.

Instead of dividing the scores themselves, a transformer divides e^{score} by its total, the *softmax*. Every weight is then positive, and a large score takes almost everything.

$$\text{weight of word } j = \frac{e^{s_j}}{\sum_l e^{s_l}}$$

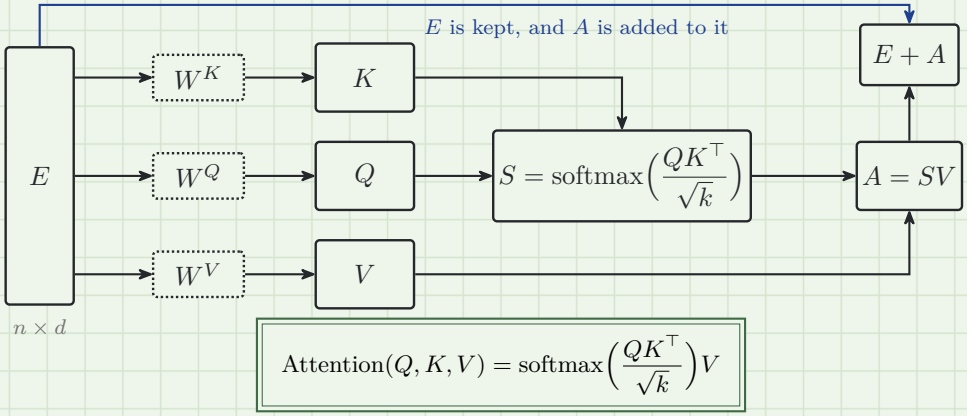
	spottet	*kat*	*winget*	*lizzard*
scores of *kat*	**0.40**	0.20	0.20	0.20
with a score of 9	**0.99**	0.00	0.00	0.00

With our scores of 0 and 1 the softmax barely chooses.

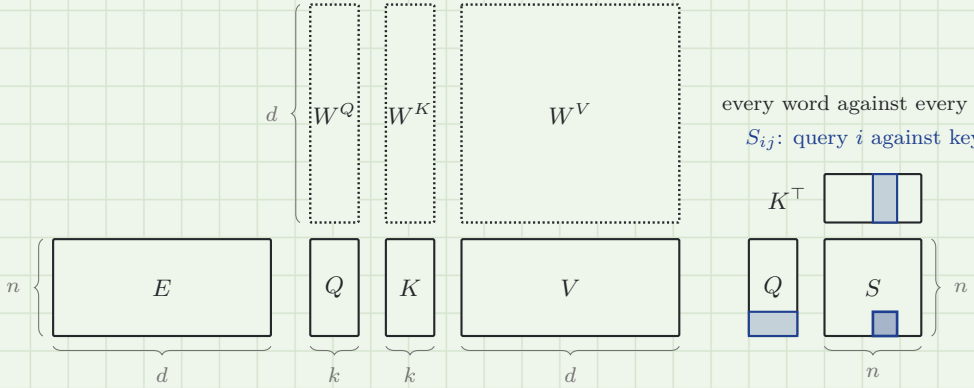
With weights three times larger, a match scores 9 and takes almost all.

A trained network learns large scores for the words that belong together. Every score is first divided by $\sqrt{k}$, so that longer queries do not make it larger. A row of zeros, an adjective's, gets 0.25 for each word: an even mix of the values, which nothing reads in this layer.

Three blocks of weights make Q, K and V from E. The softmax turns the scores into weights, the weights mix the values into A, and A is added to E.



Every block drawn to scale, with the running sentence's sizes: $n = 4$ words, $d = 9$ features, and $k = 2$ slots in a query and a key.



In a trained transformer nobody writes the weights: they are learned from text, and every head of every layer draws its own map of which words read which.

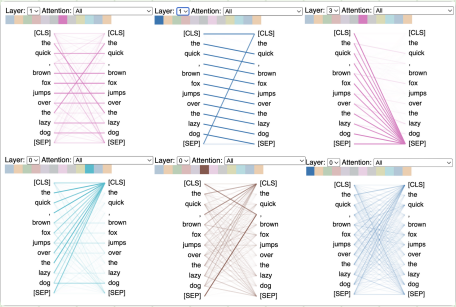


Image from https://www.comet.com/site/blog/explainable-ai-for-transformers/

One layer of attention, as we computed it on Wuggish.

1. Every word is a row of features: the sentence becomes E, $n \times d$.
2. A noun asks with a query, an adjective answers with a key: $Q = EW^Q$, $K = EW^K$.
3. The scores $S = QK^\top$ count the slots that a query and a key share.
4. Every word adds the values $V = EW^V$ it scored, $E + SV$: *spottet kat* becomes *lepperd*.
5. A transformer makes every row of scores add up to 1 with the softmax:

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{k}}\Big)V$$

Ties such as *streept kat poony* are broken by the position of every word, added to its row before Q, K and V are made.